Faster Quantum Algorithms for Vertex Cover and Independent Set through Recursive Search with Value Calls

Sebastian Senge
Independent Researcher

Abstract

We give quantum algorithms with bounded error and polynomial space, and with no quantum random access memory. Our results are these:

- A recursive search algorithm that also makes value calls to smaller instances has a quantum algorithm in time $\sqrt{B}\,\mathrm{poly}(n)$, for a classical bound B meeting three conditions.
- Vertex Cover parameterized by the solution size k can be solved in quantum time $O^*(1.1193^k)$, and in $O^*(1.1194^k)$ from a deterministic classical analysis alone.
- Maximum Independent Set on n vertices can be solved in quantum time $O^*(1.0953^n)$.

To our knowledge, the previous best quantum bounds are $O^*(1.175^k)$ [9] and about $O^*(1.1287^k)$ implied by [12, 3] for Vertex Cover, and $O^*(1.1488^n)$ [5] and about $O^*(1.1337^n)$ implied by [12, 17] for Independent Set.

1 Introduction

Many of the best exact algorithms for hard graph problems are branching algorithms, analysed by a measure on the instances. A quantum walk on the search tree [12] gives the square root of the tree size, and this applies directly when the algorithm is a plain search tree. The best classical bounds for Vertex Cover and Independent Set are not of this form. They solve a part of the graph exactly, read off its optimal value, and use this value in the rest of the search. Such a value call cannot be treated as one more branch, since its answer decides what the algorithm does next.

We show that value calls cost only a polynomial factor in total, under simple conditions on the measure. The tool is the walk on computation trees of [13], in which each vertex has its own cost. Table 1 lists the consequences.

Theorem 1.1. *Let a recursive decision procedure and a measure B satisfy the conditions (M1), (M2), (M3) of Section 3. Then the procedure has a quantum algorithm on input x_0 with error at most $1/3$, in time $\sqrt{B(x_0)}\,\mathrm{poly}(n)$, with polynomial space and no quantum random access memory. The exponent of the polynomial depends only on the constant in the conditions and on the polynomial bounds in the model.*

Theorem 1.2. *The problems of Table 1 have quantum algorithms with bounded error and polynomial space in the times listed there.*

Proposition 1.3. *The tools alone, with no value calls, give the following quantum bounds, with bounded error and polynomial space: Vertex Cover above LP in $O^*(1.5214^\mu)$, using [12, 10]; Vertex Cover of maximum degree 3 in $O^*(1.0637^k)$, using [2, 4], and in $O^*(1.0697^k)$ using [12, 10, 14]; Independent Set of maximum degree 3 in $O^*(1.0375^n)$, using [2, 4], and in $O^*(1.0410^n)$ using [12, 14]; and of maximum degree 4 in $O^*(1.0665^n)$, using [2, 4, 17], and in $O^*(1.0666^n)$ using [12, 17].*

Problem	Previous best bound, classical	Previous best bound, quantum	This paper
Vertex Cover, k	1.25281^k rand. [4], 1.25284^k [8]	1.175^k [9],	$\mathbf{1.1193^k}$ (1.1194^k)
		$\approx 1.1287^k$ implied by [12, 3]	
max degree 4	1.21103^k rand. [4], 1.21131^k [8]	$\approx 1.1024^k$ implied by [12, 8]	1.1005^k (1.1006^k)
max degree 5	1.24382^k rand. [4], 1.24394^k [8]		1.1153^k (1.1154^k)
max degree 6	1.25210^k rand. [4], 1.25214^k [8]		1.1190^k
Max Independent Set, n	1.19951^n [16]	1.1488^n [5],	$\mathbf{1.0953^n}$
		$\approx 1.1337^n$ implied by [12, 17]	
max degree 5	1.17354^n [15]		1.0834^n
max degree 6	1.18922^n [16]		1.0906^n
max degree 7	1.19698^n [16]		1.0941^n

Table 1: Time up to polynomial factors. Each base in the last column is the square root of the smallest classical base in its row, rounded up. A value in brackets uses the deterministic classical bound. An empty quantum entry means that the bound for the general problem applies. The implied bounds are explained in Remark 1.4.

The reasons are in Section 7. To our knowledge, these have not been stated before, and they are not among the rows of Table 1 since they need no new tool.

Remark 1.4. The quantum entries marked as implied are the bounds we could confirm from a plain search tree with the walk of [12]. For Independent Set, branching on a vertex of degree at least 5 gives $T(n) = T(n-1) + T(n-6)$, of base 1.28520, above the plain tree of [17]. Older algorithms have smaller bases, such as 1.2210 in [6], which would give about 1.1050^n if its separation step can be carried with a global budget. For degree 4 in k, the simple branching of [8, Proposition 28] with [17] has base 1.2152. Also, Line 2 of its algorithm Branch4 can be replaced by setting V_3 aside and solving it last, with a global budget, with a small change of the measure to pay for the bonus. In degree 4 this keeps the deterministic base of Table 1.

So Theorem 1.1 is what is used where the value calls of [15] at vertex cuts enter: degrees 5 and 6 in k, general Vertex Cover, and Independent Set of degree at least 5. We keep the level scan for Line 2 of Branch4 since it leaves the measures of [8] unchanged.

Technique. Three points make value calls cheap.

- *A cost per vertex.* In the walk of [13] the cost is governed by $\sum_v t_v^2$, for the steps t_v needed to reach v. If a node v makes value calls of cost $\sqrt{B(y)}$, the term of v is $B(y)$, so a sum of measures appears, and (M1) bounds it.
- *A gain per level.* Condition (M2) asks that $\log B(y) \le \frac{3}{4} \log B(v) + O(\log n)$. Then $B(y) \le B(v) \cdot B(v)^{-1/4} \mathrm{poly}(n)$. For nodes of large measure, the factor $B(v)^{-1/4}$ pays for the polynomial overhead of a level, so these overheads do not multiply along the $O(\log \log B)$ levels of recursion. Nodes of small measure are solved classically.
- *An order for independent parts.* An algorithm may solve a part and then the rest, and a part can hold almost all of the measure. A scan over levels of the measure tests the cheap parts first (Lemma 5.1), which gives (M2) with 1/2.

A nesting of depth $\Theta(\log n)$ also needs care with space. We reduce the error in place, by alternating projections [11], since side-by-side copies would give quasi-polynomial space.

Scope. The comparison in Table 1 is for time up to polynomial factors. The polynomial here is fixed, and large. We use the classical bounds as stated, and Section 7 argues the conditions from the structure of their analyses. The rules of [4] are generated by a program, and the bounds in brackets do not use them. Section 9 states which parts are formalized in Lean. The other parts are claims with the proofs given here. Code and Lean files are at `https://github.com/ssenge/ResearchResults`, in the folder `quantum-vertex-cover`.

2 Tools

We write O^* for a bound up to factors polynomial in n. All logarithms are natural.

Computation trees. A *computation tree* has a root and depth at most D, and each non-root vertex v is computed from its parent in t_v known steps, classical or quantum, where t_v is a function of the parent and the child index. Only leaves can be marked.

Theorem 2.1 ([13], Theorem 4)**.** *Let $T' \geq \sum_v t_v^2$. There is a quantum algorithm that uses $O(\sqrt{T'D})$ applications of a walk operator, each with $O(1)$ steps and state preparations, and accepts with probability at least $1/2$ if a marked leaf exists and at most $1/4$ if not.*

The statement in [13] has $T = \sum_v t_v^2$. A larger T' only sets a finer precision of phase estimation in its proof. Gate counts are stated there with random access gates, which are used only to reach the registers of one level of the tree. In the circuit model this costs a factor $\mathrm{poly}(D, w)$ for width w, which is enough for us.

Reversible computation. A classical computation of τ steps and space S has a reversible form of size $O(\tau^{1.585})$ and space $O(S \log \tau)$, by pebbling [1].

Error reduction in place. Let V be a unitary on a register with start state $|0\rangle$, and let p be the probability that its output qubit reads 1. Let $\Pi = V^\dagger(|1\rangle\langle 1| \otimes I)V$ and $\Delta = |0\rangle\langle 0|$. Apply r tests in turn, $\Pi, \Delta, \Pi, \ldots$, writing each outcome to a new bit. Let $b_0 = 1$, and let $X_i = 1$ if $b_i = b_{i-1}$.

Lemma 2.2. *The bits $X_1, \ldots, X_r$ are independent, with mean p.*

Proof. If $p \in \{0, 1\}$ the claim is immediate. Otherwise the plane spanned by $|0\rangle$ and $\Pi|0\rangle$ is invariant under Π and Δ. In it, let $|\pi\rangle \propto \Pi|0\rangle$, and let $|0^\perp\rangle, |\pi^\perp\rangle$ be the orthogonal vectors. Then $|\langle 0|\pi\rangle|^2 = |\langle 0^\perp|\pi^\perp\rangle|^2 = p$ and $|\langle 0|\pi^\perp\rangle|^2 = |\langle 0^\perp|\pi\rangle|^2 = 1 - p$. Before test i, whatever the outcomes so far, the state is the eigenvector of the previous projector with outcome b_{i-1}, and the start $|0\rangle$ is the case $b_0 = 1$. So test i gives $b_i = b_{i-1}$ with probability exactly p. □

This is the procedure of [11], with no witness. It uses one copy of the register and r new bits. If $p \geq 1/2$ or $p \leq 1/4$, comparing $\sum_i X_i$ with $3r/8$ errs with probability at most $e^{-r/32}$. We then copy the result and uncompute. If the error is ε, the result is within amplitude $\sqrt{2\varepsilon}$ of the exact compute and copy.

3 The model

A *recursive decision procedure* takes an instance x of size polynomial in n, with recursion depth at most $D = \mathrm{poly}(n)$. In polynomial time, from x and deterministically, it goes into one of four cases.

- **Leaf.** It answers yes or no.
- **OR.** It lists polynomially many children, and answers the OR of their answers.
- **SEQ.** It runs polynomially many *value calls* $y_1, \dots, y_s$, where y_j depends in polynomial time on the answers to earlier calls. Then it answers from these answers, or makes a *tail call* to an instance z and answers as z does.
- **Box.** It answers through a fixed unitary subroutine with bounded error, in time $\sqrt{B(x)}\,\mathrm{poly}(n)$ and polynomial space.

An optimization value is reached through polynomially many thresholds on a budget in the instance. A maximum with offsets, $\alpha(x) = \max_i(\alpha(x_i) + c_i)$, is an OR in this form.

The *OR-tree* of x is the tree reached from x through OR edges and tail edges. The *full call tree* of x also has the value edges. A *measure* is a function $B(x) \geq 2$, computable in polynomial time, with $\log B \leq L = \mathrm{poly}(n)$.

Definition 3.1. For a constant $C \geq 1$ and large n:

(M1) For x the input or the target of a value call, and for every antichain X in the OR-tree of x, $\sum_{a \in X} B(a) \leq n^C B(x)$.

(M2) For every value call y at a node v, $\log B(y) \leq \frac{3}{4} \log B(v) + C \log n$.

(M3) For every node x, the number of nodes of the full call tree of x, plus the sum of $B(a)$ over its Box nodes a, is at most $n^C B(x)$.

Condition (M3) counts what a classical run counts: the nodes, and $B(a)$ for a Box whose subroutine is a randomized one with success probability $1/B(a)$. It cannot be dropped. Without it, $B = 2$ at every node of a balanced binary tree of value calls, with Box nodes at its leaves, satisfies (M1) and (M2), and it would compute the parity of exponentially many bits with polynomially many queries.

Classical analyses usually give a local inequality $\sum_c B(c) \leq B(r)$, sometimes over a constant number of levels of branching, or with an amortized bonus. These give (M1) as follows.

Lemma 3.2 (macro steps)**.** *Let the OR-tree of x be cut into macro steps, with x a root. Let each macro step have a root r and at most s nodes, each of measure at most $KB(r)$, and let its lower boundary consist of roots c of further macro steps with $\sum_c B(c) \leq B(r)$. Then every set X of nodes has $\sum_{a \in X} B(a) \leq (D+1)sKB(x)$.*

Proof. By induction the roots at fixed macro depth j have total measure at most $B(x)$. Each node lies in the macro step of one root, and a macro step holds at most s nodes, each of measure at most K times its root. So the nodes at macro depth j contribute at most $sKB(x)$. Sum over $j \leq D$. □

4 The algorithm

Let $q \geq 1$ be a constant such that n^q bounds the factor $(D+1)n^C$, the cost of a node or of a Box at reduced error times n^C, the number of children and calls of a node, the width of one level of the recursion, and the overhead of the walk with its error reduction. We choose

$$W = 22q + 3, \qquad A = 7q + 4W + 1,$$

and take n large against W and A. Levels m are real numbers, computed to polynomial precision and rounded up, which changes the bounds below by a factor $1 + o(1)$.

Algorithm 1 $\textsc{Solve}_m(x)$, for an input or value-call target x with $\log B(x) \le m$

1: **if** $m \le W \log n$ **then**
2: Traverse the full call tree of x classically, with each Box at reduced error, in reversible form.
3: **else**
4: Run the walk of Theorem 2.1 on the OR-tree of x, with $T' = e^m(2n^{3q} + n^{5q+4W} + n^{A+5q-(W-q)/3})$. A step to a child of a node v is:
OR node: compute the child, in polynomial time.
SEQ node: run the value calls in order with $\textsc{Solve}_{m_v}$, $m_v = \frac{3}{4}\log B(v) + q\log n$; the child is
the tail z, or a leaf marked by the answer.
Box node or Leaf: a leaf child marked by the answer of the Leaf, or of the Box at reduced error.
5: Reduce the error to $\exp(-4L - n)$ in place, by Lemma 2.2.

Each child is computed, its description copied and the rest uncomputed. So only the descriptions of the nodes stay on the path, and the running call uses one ancilla register, shared by the levels of the tree, since it is clean between edges and only one edge is in progress. Every value call at v runs the same circuit $\textsc{Solve}_{m_v}$, and the same number of steps, whatever the target y is. By (M2) this level covers y. So the number of steps and the gate at each step depend only on the parent and the child index, as the known times in Theorem 2.1 require, and a step can be applied with the moment in superposition. Marked vertices are leaves added below the nodes, so the root is unmarked. Vertex states remain orthonormal through their labels, namely the path, the child index and the moment.

Proof of Theorem 1.1. Let $Q(m)$ be the size of $\textsc{Solve}_m$. We show $Q(m)^2 \le n^A e^m$ for $m \ge 0$. The theorem follows at $m = \log B(x_0)$.

Base. For $m \le W \log n$, (M3) gives a traversal of $\tau \le n^{q+W}$ steps, counting $\sqrt{B(a)}$ for each Box a. Its space, before pebbling, does not depend on W. Pebbling gives size $\tau^{1.585}$ up to logarithmic factors, and multiplies the space by $\log \tau \le (q+W)\log n$, which is polynomial with an exponent not depending on W for n large against W. As $0.83W \ge 3.17q$, this gives $Q(m)^2 \le n^{q+4W}$.

Step. Let $m > W \log n$. Let v be a node of the OR-tree of x. Applying (M1) at each depth gives

$$\sum_v B(v) \le n^q e^m,$$

and as $B \ge 2$ the number of nodes is at most the same. Also $\log B(v) \le m + q\log n$. A child c of v has

$$t_c^2 \le n^q\big(1 + n^q Q(m_v)^2 + B(v)\big),$$

with the last term only for a Box. If $m_v \le W \log n$ then $Q(m_v)^2 \le n^{q+4W}$. Otherwise $\frac{3}{4}\log B(v) \ge (W-q)\log n$, which gives $B(v)^{-1/4} \le n^{-(W-q)/3}$. Moreover $m_v \le \frac{3}{4}m + 2q\log n < m$, as $W > 8q$. So the induction applies, and

$$Q(m_v)^2 \le n^A e^{m_v} \le n^{A+q} B(v)^{3/4} \le n^{A+q-(W-q)/3} B(v).$$

Sum over the at most n^q children of each node. This gives the bound

$$\sum_c t_c^2 \le e^m\big(2n^{3q} + n^{5q+4W} + n^{A+5q-(W-q)/3}\big) = T',$$

where the first term collects the 1 and the Box term. The walk with its error reduction has size at most $n^q\sqrt{T'}$, so

$$Q(m)^2 \le e^m\big(2n^{5q} + n^{7q+4W} + n^{A+7q-(W-q)/3}\big).$$

Since $(W-q)/3 = 7q+1$ and $A = 7q+4W+1$, each term is at most n^{A-1}, and $Q(m)^2 \le 4n^{A-1}e^m \le n^A e^m$. The constants A and W enter the circuit only through counters of logarithmic length and through the size of n, so q does not depend on them.

Errors. A step of a value call or a Box that errs with probability ε is within amplitude $\sqrt{2\varepsilon}$ of the exact step. The walk operators are block diagonal in the path and the child index, so the error of one use is a maximum over branches, and it is at most $e^{-2L-n/2}$ up to constants. The number of uses in one run of the walk is at most $e^L\mathrm{poly}(n)$, so the total error of a run is negligible, and $\textsc{Solve}_m$ then reduces its error afresh with $r \ge 32(4L+n)$. So nothing accumulates across levels.

Space. Only one call runs at a time. A level holds the descriptions on a path, the walk registers and the r bits of Lemma 2.2, all polynomial, and one lower level. With (M2) there are $O(\log L)$ levels above the base, so the qubits are polynomial. □

Witnesses. For Vertex Cover, a cover comes from self-reduction: for each vertex v in turn, decide $(G-v, k-1)$, and keep the change on a yes. The measures in Section 7 do not increase by more than a constant factor on this step, and n calls suffice. The same holds for Independent Set through the complement of the cover.

Remark 4.1. Any constant below 1 works in place of 3/4 in (M2). The base case cannot be left out: with no classical base, the polynomial factors of the levels near $\log n$ do multiply. A polynomial factor that grows with the number of value calls per node is also needed in general, as parity trees show.

5 Independent parts

An algorithm may solve independent parts of a graph one after the other. In the order of the classical analysis, a part may hold almost all of the measure, which violates (M2). The following scan fixes the order.

Let a node v with budget k be a disjoint union of parts $P_1, \dots, P_\ell$, with answer yes iff the minimum budgets c_i of the parts satisfy $\sum_i c_i \le k$. Let $g_i(b) \ge 0$ be a computable bound on the log measure of P_i with budget b, nondecreasing in b, with $g_i(b) = 0$ for b below the LP value of the part, where the test answers no from the LP. Suppose there is a constant c_0 such that the first inequality below holds whenever $\sum_i c_i \le k$, and the second whenever $\sum_{j\ne i} c_j \le k$:

$$\sum_i g_i(c_i) \le \log B(v) + c_0, \qquad g_i\Big(k - \sum_{j\ne i} c_j\Big) \le \log B(v) + c_0 \ \text{ for each } i. \tag{1}$$

Lemma 5.1 (level scan)**.** *There is a procedure with polynomially many value calls, each of log measure at most* $\frac{1}{2}(\log B(v) + c_0)$*, and at most one tail call of log measure at most* $\log B(v) + c_0$*, with the same answer as* v*.*

Proof. Sort the values $g_i(b)$, for $0 \le b \le k$, into levels $m_1 \le m_2 \le \cdots$. While at least two parts are unsolved, take the next level m. If no level is left, or if $2m > \log B(v) + c_0$, answer no. Otherwise, for each unsolved part, as long as at least two are unsolved, test the budgets b with $g_i(b) \le m$ in increasing order, and if one succeeds, the part is solved with $c_i = b$. When one part is left, make the tail call with the remaining budget. If the remaining budget is negative at any time, answer no.

Every test has log measure at most m, so at most $\frac{1}{2}(\log B(v) + c_0)$. Suppose the instance is feasible and the procedure answers no at level m. Two parts are unsolved, so each has $g_i(c_i) > m'$ for the previous level m', and hence $g_i(c_i) \geq m$. Thus $2m \leq \sum_i g_i(c_i) \leq \log B(v) + c_0$, by (1), a contradiction. A no on an infeasible instance is correct, and the tail is bounded by (1). □

6 Randomized subroutines

Theorem 6.1. *Let a one-sided randomized algorithm follow a path of polynomial length, answering yes with probability at least $1/B(x)$ on a yes instance and never on a no instance. Then it has a quantum algorithm in time $\sqrt{B(x)}\,\mathrm{poly}(n)$, with bounded error, polynomial space and no quantum random access memory.*

Proof. The coins form a register, and the path is computed reversibly in polynomial time and space. Amplitude amplification [2] with a uniform number of iterations below $O(\sqrt{B(x)})$ finds an accepting path with constant probability, and a no instance has none. Rounding the probabilities of the path to polynomially many bits changes the success probability by a constant factor. □

Lemma 2 of [4] gives such a path, with $B = 2^{\mu(I)}$ for its measure μ. This gives a Box in the sense of Section 3.

7 Applications

We now argue the conditions of Theorem 1.1 for the algorithms behind Table 1, which gives Theorem 1.2, and the reasons for Proposition 1.3. The recurrences themselves are those of the sources, and we do not repeat them. In each case, the classical analysis proves a running time bound by induction over a measure, which gives (M3) outside the scan, and it gives (M1) through Lemma 3.2, since its branch sequences have constant depth and its bonuses are bounded.

Vertex Cover in k [8]. The measure is $\phi = a\mu + bk$ with a bounded bonus, for $\mu = k$ minus the LP value. Algorithm Branch4 has one value step. Its Line 2 solves the subcubic part V_3, of minimum budget k_3, using the cheaper of a bound in μ and a bound in n, and continues on the rest. Let g_1 be the logarithm of the cheaper bound for V_3, and $g_2 = \phi$ for the rest. The analysis of Line 2 rests on the inequality

$$\min\{\mu \log 2.3146,\ 2(k-\mu)\log 1.083506\} \leq a\mu + bk,$$

valid for all $\mu, k \geq 0$, here applied to the parameters of V_3. The quantity μ is additive over independent parts, as k and the LP value are. As G is simplified, the LP value of V_3 is $n_3/2$, and each part has $\mu \geq 0$ at its c_i. So (1) holds, with c_0 covering the bonus, and Lemma 5.1 gives (M2). The tail needs no constant: a solved part has budget $c \geq 2$, so the tail has log measure at most $a\mu + bk - bc \leq \phi$, as $2b \geq \alpha$, and at Line 2 the bonus is α. If V_4 is empty, the tail on V_3 exceeds $B(v)$ by at most e^{α}. It then leaves Branch4 for good, so Lemma 3.2 applies on each side of this switch, with a constant factor.

For (M3) at a node with a scan, let h be the last level used and s the number of vertices. The parts are disjoint, and $e^{g_i(b)}$ grows geometrically in b, or is constant on a subcubic part that does not return to Branch4. So the tests cost $O(s^c e^h)$, the tail has at most $s-1$ vertices and log measure at most $\log B(v) + c_0 - h$, and $2h \leq \log B(v) + c_0$. Induction on s gives $T(v) \leq s^c B(v)$ for a fixed c.

All other branch algorithms of [8] make tail calls only. Where the source dovetails two algorithms, we choose the one with the smaller computable bound, which is enough since the analysis uses only the minimum of the bounds. The same holds for its Lemma 15, whose bound $d = 2c(a+b)/(a+2c)$ is homogeneous of degree 1 in (a, b, c), so taking square roots of the bounds gives the square root of the result.

Vertex Cover above LP [10]. The value step in the source is the separate treatment of connected components. Here μ is additive over components, so the budget can stay global. Branching inside one component gives the same drops of μ, and components of bounded size take constant time. This is a plain search tree with the same recurrences, which gives the first bound of Proposition 1.3 through [12], and it is how this algorithm is used below Branch4.

Maximum Independent Set [14, 17, 15, 16]. The algorithms for degree 3 and 4 make calls of constant size, on components or reducible vertex sets, or on line graphs, all polynomial, so they are search trees for our purposes. A maximum with offsets is an OR with thresholds. This gives the bounds of Proposition 1.3 from [14, 17].

The degree-5 algorithm has steps with value calls at a vertex cut, on $G[V_1]$ and its variants, followed by a tail, with the recurrences $C(\mu) \le 2C(x)+C(\mu-x)$ and $C(\mu) \le 4C(x)+C(\mu-x)$, for x the measure of V_1, and with a shift of at most 0.175. We take V_1 as the side of smaller measure, which the divide-and-conquer theorems of the source allow, so $x \le \mu/2$. The form $4c^{x+0.175} + c^{\mu-x+0.175} \le c^{\mu}$ is convex in x, so it is enough to check it at the ends of the range $[w_3, \mu/2]$, with $w_3 > 0.5$. At $c = 1.17$, it holds for $\mu \ge 28.5$, and below this μ is constant. Value calls have measure at most $x + 0.175 \le \frac{3}{4}\mu$ for $\mu \ge 0.7$, so (M2) holds, and the tail has measure at most μ. The algorithms for degree 6, 7, 8 and general graphs make tail calls, and use the lower-degree algorithms as tails.

Randomized rows [4]. The subcubic rules are one-sided paths, so they give the degree-3 bounds through Theorem 6.1. Its Lemma 2 with weights 1 turns the tree of [17] into such a path over them, which gives the degree-4 bound in n. The rows for k take the frameworks of [17, 8], with the subcubic and degree-4 randomized algorithms in place of the deterministic ones. By Theorem 6.1, these are Box nodes in the sense of Section 3, and Box nodes reached by a value call are solved as in Algorithm 1. So the bounds of its Table 1 carry over, with measures $a_i\mu + b_i k$ as listed in its Lemma 18. Its degree-4 framework is used as stated.

8 Measurements

The scripts in `sim/` test the walk and the conditions on small search trees, with a plain branching rule for Vertex Cover as a stand-in for the rules of the sources. They do not test the bounds of Table 1, and nested walks are not run as circuits.

- The walk with costs $t_v = 1 + B(v)^{3/8}$ runs on 27 search trees by exact linear algebra. With a marked leaf, the root has weight at least 0.500 on eigenvalue 1. Without one, its weight on phases $|\theta| \le \varepsilon$ stays within $\varepsilon^2(1 + DT)$, and reaches 0.994 of this bound.
- Phase estimation runs as Qiskit circuits in Aer, noiseless, at up to 13 qubits, with dense unitaries computed from the tree. The 8 instances with a cover accept with probability at least 0.742, and the 8 without at most 0.040.
- For $B = x^{\text{budget}}$ on 36 trees, the largest antichain sum over $B(\text{root})$ is 1.000 at $x = 1.4656$, the root of the branching vector $(1, 3)$, and it reaches 9.566 at $x = 1.1$.

- The level scan in its budget form, on 180 budgets on disjoint unions, gives the same answer as brute force every time.
- The recursion of the costs, with $q = C = 1$ and $W = 25$, keeps the exponent of Q^2/B at most 55.25 up to $B = n^{800}$. Lemma 2.2 holds by exact enumeration of outcome strings, to within 10^{-16}.

9 Formal verification

The folder `lean/` builds with Lean 4 and Mathlib, with no `sorry`, and only the standard axioms in all 78 theorems listed in `Axioms.lean`. It proves Lemma 3.2, the semantics of the procedure (the answer is yes iff the OR-tree has a marked leaf), the recursion of the costs with the constants above, the level scan in its budget form with its correctness and the bound on the levels, the two-dimensional fact behind Lemma 2.2 with the product form, and the arithmetic of Table 1.

The walk, the error reduction, the amplitude error and pebbling are named hypotheses, as inequalities on real functions, with no circuit model, and a satisfiability theorem shows that they are consistent. (M1), (M2), (M3) are fields of a structure of conditions.

Not formalized are the circuits, gate counts and the absence of random access memory; the walk itself, the known-times oracle and the shared register; the block decomposition of two projectors and the tail bound; pebbling; the link between Lemma 3.2 and (M1) in their two formal forms; the level form of the scan with (1); and all of Section 7.

10 Limits and open questions

- The polynomial factor is fixed but large, and there is no gate count. So these bounds say nothing about instances that fit on current devices.
- The classical bounds are used as stated. Section 7 argues the conditions at the level of the structure of the algorithms, and the individual recurrences are taken from the sources. For degrees 3, 4 and 5 we use the technical report versions.
- Rules generated by a program, in [4], are taken as stated.
- Comparison with [7], which treats recursive decomposition by nested search, and with [9], is based on their stated bounds.
- Can (M2) be weakened to $\log B(y) \le \log B(v) - \omega(\log n)$? The gain in the proof would then shrink with the level.
- Can the polynomial factor be made small for the concrete algorithms?
- We prove no lower bound.

Use of AI

The proofs, the code and the Lean files were developed with Claude Opus 5.5 (Anthropic), which was also used to search the literature, to check the proofs and to draft the text. The author is responsible for the content.

References

[1] C. H. Bennett. Time/space trade-offs for reversible computation. SIAM Journal on Computing 18, 1989.

[2] G. Brassard, P. Høyer, M. Mosca, A. Tapp. Quantum amplitude amplification and estimation. Contemporary Mathematics 305, 2002, arXiv:quant-ph/0005055.

[3] J. Chen, I. A. Kanj, G. Xia. Improved upper bounds for vertex cover. Theoretical Computer Science 411, 2010.

[4] K. Clinch, S. Gaspers, T. Z. He, S. Mackenzie, T. Zhang. A faster randomized algorithm for Vertex Cover: an automated approach. arXiv:2510.09027.

[5] S. Dörn. Quantum complexity bounds for independent set problems. arXiv:quant-ph/0510084.

[6] F. V. Fomin, F. Grandoni, D. Kratsch. A measure & conquer approach for the analysis of exact algorithms. Journal of the ACM 56(5), 2009.

[7] M. Fürer. Solving NP-complete problems with quantum search. LATIN 2008, LNCS 4957.

[8] D. G. Harris, N. S. Narayanaswamy. A faster algorithm for Vertex Cover parameterized by solution size. STACS 2024, arXiv:2205.08022.

[9] A. Jeļisejevs. Bachelor's thesis, University of Latvia, 2021. Bound $O(1.175^k k^{O(1)} + n\sqrt{m})$ as stated in its abstract.

[10] D. Lokshtanov, N. S. Narayanaswamy, V. Raman, M. S. Ramanujan, S. Saurabh. Faster parameterized algorithms using linear programming. ACM Transactions on Algorithms 11(2), 2014, arXiv:1203.0833.

[11] C. Marriott, J. Watrous. Quantum Arthur-Merlin games. Computational Complexity 14, 2005.

[12] A. Montanaro. Quantum-walk speedup of backtracking algorithms. Theory of Computing 14, 2018, arXiv:1509.02374.

[13] J. Vihrovs. Quantum search on computation trees. arXiv:2505.22405.

[14] M. Xiao, H. Nagamochi. Confining sets and avoiding bottleneck cases: a simple maximum independent set algorithm in degree-3 graphs. Theoretical Computer Science 469, 2013.

[15] M. Xiao, H. Nagamochi. An exact algorithm for maximum independent set in degree-5 graphs. Discrete Applied Mathematics 199, 2016.

[16] M. Xiao, H. Nagamochi. Exact algorithms for maximum independent set. Information and Computation 255, 2017, arXiv:1312.6260.

[17] M. Xiao, H. Nagamochi. A refined algorithm for maximum independent set in degree-4 graphs. Journal of Combinatorial Optimization 34, 2017.